Automatiser les MEPs

Se simplifier la vie avec Github Actions

Axel Mathieu-Le Gall · Senior Fullstack Developer · Wealthcome

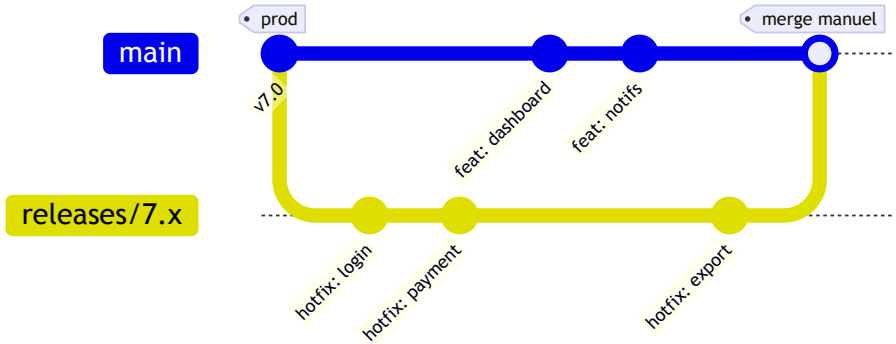
45 minutes pour une mise en prod.

3 personnes mobilisées.

Aujourd'hui : **4 minutes**.

1 clic et c'est tout

Le workflow qu'on avait hérité



- ❌ **Preprod** = env de test + env de démo
- ❌ **Hotfix** qui ne remontent pas dans `main`
- ❌ Quelqu'un joue les **plombiers Git** à la place des devs

Les 3 questions qu’on se posait toutes les semaines

🔀 "Est-ce que j’ai bien pris tous les hotfix ?"

Est-ce que j’ai bien géré les conflits ? Est-ce que je perds un fix qu’un·e collègue a fait il y a 3 jours ?

📢 "C’est quoi qui part en prod ce soir ?"

Changelog = liste brute de titres de commits. Pas de nomenclature, pas toujours de ticket. Statuts MàJ à la main.

🕰️ "Ça a été déployé quand, exactement ?"

Rythmes de préprod et de prod irréguliers. On déployait "quand on pouvait". Zéro prévisibilité.

Ce qu’on a remis en place d’abord

L'automatisation n'a pas été la 1ère étape. Il a fallu d'abord réparer les fondations.

✅ Un flow Git simple et redevable

GitHub Flow — `main` stable + branches `feat/*`, `fix/*`. Plus de `releases/X.x` orphelines.

✅ Des environnements dédiés et étanches

`dev` → `staging` (feature branch) → `preprod` (tests produit) → `prod`
`staging` pour les feature branch

✅ Une CI qui bloque avant la préprod

Tests, lint, build reproductibles, nomenclature PR

La partie web : 6 repos, 1 équipe

🎨 web	🎨 cgp-platform	⚙️ backend	⚡ aggregated	🗄️ shared-aggregations
Front B2C · React/TS	Front B2B · React/TS	API · Nest/TS	Nouvelle API · Hono/TS	Migrations · Schémas DB
🚀 déployé	🚀 déployé	🚀 déployé	🚀 déployé	🚀 déployé

📦 **wealthcome-cgp** Types partagés ↑ web · cgp-platform · backend

🗄️ **shared-aggregations** ↑ backend · aggregated

Ce qu'on vivait au quotidien :

- 🐌 Déployer une feature cross-app = orchestrer ces repos, à la main, dans le bon ordre
- 🔀 Taguer, release, notifier Slack = 5× le même process, manuellement
- 📋 Savoir ce qui part en prod ce soir = checker 5 GitHub, recopier à la main

Ce qu’on cherchait à résoudre

🎯

Un point d’entrée unique

Une équipe ≠ se disperser entre 3 repos. Où cliquer, où déclencher, où regarder ?

🔗

De la logique partagée

Tagging, release, notifs : 3× la même chose → une seule implémentation, réutilisable.

📊

Une vue d’ensemble

Ce qui part en prod ce soir, dans toutes les apps → agrégation, pas dispersion.

Pas de refonte d’architecture. Juste une couche d’orchestration.

L’inspiration : le package `meta`

Créé par @mateodelnorte · Dernière MAJ : 2020

> "Why choose many repos or a monolithic repo, when you can have both with a meta repo?"

L’idée qu’on a gardée

- 📄 Un descripteur listant tous les repos
- 🎯 Un repo "parent" qui orchestre
- ⚡ Des commandes qui s’appliquent partout

Pourquoi notre propre implémentation

- Package non maintenu depuis 5 ans
- On voulait GitHub Actions + pnpm workspace
- Besoins spécifiques : Notion, Slack, tags RC/prod

---

pnpm workspace + GitHub Actions + scripts bash = même philosophie, stack 2026

`wealthcome-meta` : la structure

```
wealthcome-meta/
│
├── apps/                       # repos clonés (gitignored)
│   ├── backend/
│   ├── cgp-platform/
│   └── web/
│
├── packages/                   # logique métier partagée
│   └── generate-changelog/     # (TypeScript)
│
├── scripts/                    # orchestration locale
│   ├── sync-repos.sh
│   ├── add-repo.sh
│   └── generate-*-tag.sh
│
├── .github/                    # CI/CD partagée
│   ├── actions/
│   └── workflows/
│
├── repos.json                  # 👈 le descripteur
└── pnpm-workspace.yaml
```

Le dev workflow en 2 étapes

1. LE DESCRIPTEUR

```
[
  {
    "name": "backend",
    "url": "git@github.com:..."
  },
  {
    "name": "cgp-platform",
    "url": "git@github.com:..."
  },
  {
    "name": "web",
    "url": "git@github.com:..."
  }
]
```

repos.json

2. ONBOARDING EN 1 COMMANDE

```
$ git clone .../wealthcome-meta
$ cd wealthcome-meta
$ pnpm install
$ pnpm sync

✓ Cloned backend
✓ Cloned cgp-platform
✓ Cloned web
```

terminal

Avant : 1 journée d'onboarding · Après : **2 minutes**

Un pipeline en 2 niveaux

La décision architecturale clé.

Guérir la douleur du hotfix

🔀 *"Est-ce que j'ai bien pris tous les hotfix ?"*

1 · VÉRIFIER LA SYNCHRONISATION

```yaml
- name: Check hotfix sync
  run: |
    missing=$(git log main..releases/X.x \
              --oneline)
    if [ -n "$missing" ]; then
      echo "::error::Hotfix non remontés"
      exit 1
    fi
```

2 · VALIDATION HUMAINE

- Le meta crée la release en **DRAFT**
- Un humain relit, valide, publie
- Le push du tag déclenche le déploiement

→ 2 filets de sécurité avant toute MEP

Guérir les rythmes irréguliers

> 🕐 *"Ça a été déployé quand, exactement ?"*

PRÉPROD — TOUS LES MATINS

```yaml
on:
  schedule:
    - cron: '0 7 * * 1-5'
    #        ↑ 9h, jours ouvrés
```

Ce qui a été mergé hier est testable en arrivant au bureau.

PROD — 2× PAR SEMAINE, À LA DEMANDE

```yaml
on:
  workflow_dispatch:
    inputs:
      repos:
        description: 'all OR csv'
        default: 'all'
```

Mardi et jeudi. L'équipe et le produit s'y préparent.

Fini le "quand on pourra". Tout le monde sait quand.

Guérir les communications

> 📢 *"C'est quoi qui part en prod ce soir ?"*

1 · Nomenclature forcée à la PR

```yaml
uses: wealthcome-SAS/actions/check-pr-title@stable
# Pattern imposé : feat(module): description [TICKET-42]
```

→ Pas de ticket = pas de merge

2 · Changelog enrichi, pas brut

```
packages/generate-changelog/
├── github.ts     ← fetch PRs
├── notion.ts     ← enrichit avec les titres tickets
└── slack.ts      ← formate + @tag les owners
```

→ Produit lit "Ajout export PDF AUTH-42", pas "fix: update flow (PR #312)"

3 · Tags @personnes + MAJ Notion automatique

Ticket mentionné → owner @tag dans Slack · statut Notion passe à "Déployé" · plus de DM "c'est parti ?"

Le dry run — l’option qui donne confiance

```yaml
workflow_dispatch:
  inputs:
    dry_run:
      description: 'Simulation sans effet de bord'
      type: boolean
      default: false
```

CE QUE ÇA FAIT

- ✅ Simule : check_diff, calcul du tag, changelog
- ✅ Affiche dans les logs ce qui serait fait
- ❌ Ne crée pas de tag
- ❌ Ne poste rien sur Slack
- ❌ Ne met rien à jour dans Notion

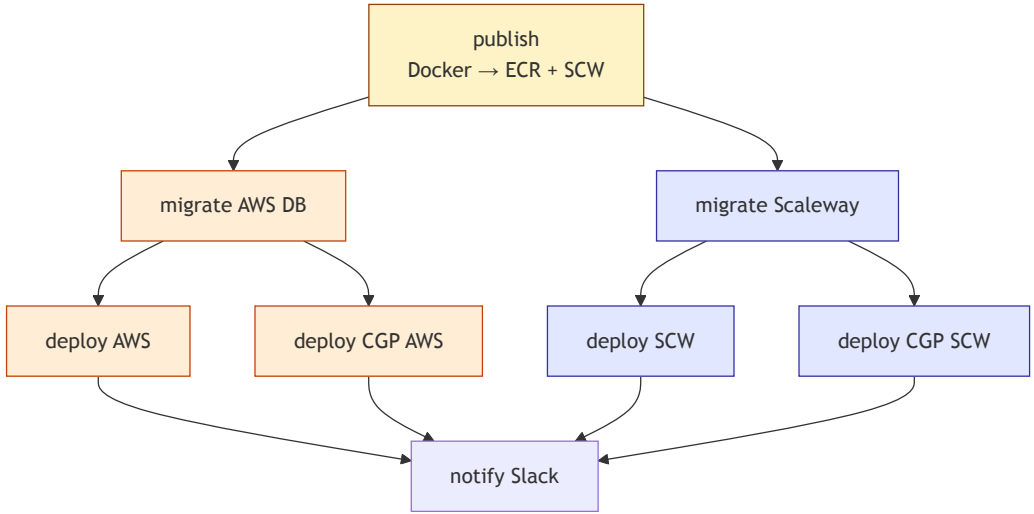
QUAND ON L'UTILISE

- Avant une release sensible (vendredi)
- Vérifier la nomenclature des PRs
- Former un nouvel arrivant sans risque
- Débugger quand on a peur de casser

💡 *La meilleure automatisation, c’est celle qu’on ose lancer.*

Le flow backend : 2 clouds, 1 tag

Déclencheur : `on: push: tags: [v*.*.*-rc.*]`



3 choses à retenir : pattern de tag · DAG des `needs:` · `concurrency:` groups

Runners Kube + actions partagées

🖥️ Runners self-hosted (EKS)

Pourquoi :

- Accès réseau direct aux clusters
- Caches partagés entre jobs
- Coût (pour un volume élevé)

```
migrate:
  uses: ./.github/workflows/migrate.yml
  with:
    runner: preproduction-services
    environment: preproduction-aws
```

📦 Actions partagées

Repo `wealthcome-SAS/actions` :

```
- uses: wealthcome-SAS/actions/send-slack-message@stable
- uses: wealthcome-SAS/actions/check-pr-title@stable
- uses: wealthcome-SAS/actions/.github/workflows/publish
```

→ DRY à travers tous nos repos

→ Une correction = tous les repos l'ont

Le pipeline en action

Et si je vous montrais ?

github.com/wealthcome-SAS/wealthcome-meta · Actions · prerelease.yml

Du clic "Run workflow" au "✅ en prod" : ~4-5 minutes

Le vrai blocker, c’était pas la technique

DÉCOR

Une équipe en place. 3 ans de process installé. Moi, nouvel arrivant, qui arrive avec ma solution.

DRAME

2 mois de friction. Meetings qui tournent au vinaigre. Un senior qui me dit : *"On a toujours fait comme ça."*

LEÇON

à remplir : ton vrai déclic — un pair qui t’a rejoint ? Un petit succès visible ? Une démo concrète ?

Quand le tag a pointé trop loin

DÉCOR

L’automatisation tourne depuis plusieurs semaines. L’équipe lâche prise, fait confiance. Les releases s’enchaînent sans accroc.

DRAME

Une release part. Le tag est créé automatiquement… mais il pointe **plus loin** que prévu dans l’historique. Des commits non prêts se retrouvent en prod.

IMPACT

Le vrai coût : un process isolé

Le bug technique était gérable. Le vrai problème était ailleurs.

COMMENT ON A RECONSTRUIT LA CONFIANCE

📢

1 · Présentations aux équipes tech + produit

Rendre l’opaque visible

🔄

2 · Réunions d’itération avec les leads

Co-construire, pas imposer

CE QU'ON A RETENU

Pas discipline *ou* automatisation.

Discipline pour **décider**.

Automatisation pour **exécuter**.

Tout ce qui dépasse 5 minutes de répétitif mérite d'être automatisé.

1 mois plus tard

6 → 7	45m → 4m	3 → 1
MEPS PROD / SEMAINE	DURÉE D'UNE MEP	PERSONNES MOBILISÉES

~14 heures d'équipe gagnées par semaine

Pipeline prod **18min → 12min** (-33%) · web **33min → 17min** (-48%)

Préprod **×1,6** — 15 → 25 déploiements/semaine

0 hotfix perdu depuis le déploiement

Ce que l’équipe en dit

photo

Solène Oruezabal — Équipe produit

"Depuis le changement, c’est clairement la charge mentale qui a changé. Je sais que c’est automatique — plus besoin de relancer 25 fois les équipes dans la journée."

photo

Rémi Poulenard — Developer · Wealthcome

"La mise en place de notre nouveau processus d’automatisation a profondément transformé notre manière de travailler. Les déploiements, désormais simplifiés, génèrent moins de conflits et mobilisent moins de temps de gestion. Résultat : une équipe plus confiante et une vélocité de livraison nettement améliorée."

photo

— Lead Tech

Je release : `@{handle}/meta`

🎯

Opinionated

pnpm workspaces + GitHub Actions + release orchestrées

🚀

Out-of-the-box

Workflows de release, tagging RC/prod, notifs Slack

🔌

Extensible

Hooks pour brancher Notion, Linear, Jira ou custom

`npm i -D @{handle}/meta` 🔗 github.com/{handle}/meta 🧪 alpha · stable prévu {date}

Un repo modèle pour commencer

🔗 github.com/{handle}/meta-examples

📂

monorepo-example

- Turborepo
- React (front)
- Fastify (back)
- Contrats partagés
- Déploiement Vercel + Fly.io

📂

metarepo-example

- pnpm workspaces
- `@{handle}/meta` (releases)
- 3 apps clonées via repos.json

← *notre approche*

💡 *Le code du talk, littéralement.*

→ 2 setups prêts à cloner · CI/CD fonctionnelle · Documentation étape par étape

Merci

↑ Ton retour sur ce talk (2 min)

Axel Mathieu-Le Gall

Senior Fullstack Developer · Wealthcome

💼 https://linkedin.com/in/axel-mathieu-le-gall-361b1510a

💻 https://github.com/AxelMth